VLSI Implementation of AlphaEvolve Based Rank-48 Algorithm for 4×4 Real-Valued Matrix Multiplication

Ettore Napoli
DIEM
University of Salerno
Salerno, Italy
enapoli@unisa.it

***Abstract*—Matrix multiplication circuits are widely used as accelerators in 3D graphics, communications, artificial intelligence, and other domains. Recent years have seen significant advances in efficient algorithms for small-dimension matrix multiplication. The AlphaEvolve algorithm for complex-valued matrices achieves rank-48 using only 48 binary multiplications, improving on the Naive algorithm (64 multiplications) and the previous best-known methods (49 multiplications). Following this development, a new algorithm for real-valued 4×4 matrix multiplication was proposed in June 2025. This work presents the first hardware implementation of the algorithm benchmarked against the Naive algorithm commonly employed in commercial accelerators. Synthesized in a 14 nm FinFET standard cell library, the proposed circuit requires fewer hardware resources than the Naive implementation for input bit-widths above 40 bits. While the Naive implementation achieves higher speed and lower power, the performance gap decreases with increasing operand size, highlighting the potential of the proposed approach for high-precision applications.**

***Index Terms*—Hardware architecture, matrix multiplication, VLSI, AlphaEvolve algorithm, Strassen's algorithm.**

I. Introduction

The explosion of big data, communication, and artificial intelligence leads to an increase in the demand of computational power. Most of the computational workload comes from matrix multiplications, driving research in the field of general matrix-matrix multiplication (GEMM).

Advancements in the field come from tailoring and optimizing the implementation for different processing platforms [1]–[7] and from the research of more efficient algorithms.

The Naive approach, requiring n^3 multiplications to compute the product of two $n \times n$ matrices, is widely used due to its regularity that easily integrate with memory architectures but is not optimal in terms of complexity. In 1969, Strassen introduced an algorithm with lower asymptotic complexity that in the specific case of 4×4 matrix multiplication requires 49 multiplications instead of the 64 needed by the Naive approach [8]. Subsequent improvements to the Strassen's algorithm have been proposed by Winograd in 1971 [9] and by other researchers in 2017 and 2023 [10], [11]. The improved algorithms manage to reduce the number of add/subtract operations while maintaining the number of multiplications.

In 2025, the AlphaEvolve framework, [12], [13], introduced a groundbreaking algorithm designed for complex-valued 4×4 matrix multiplication that reduces the number of multiplications to 48 offering the first improvement, after 56 years, over Strassen's algorithm. The findings have been used in Google's compute stack optimizing matrix-multiplication kernels [12]. A SW verification of the algorithm is at [14].

In June 2025, Dumas et al. [15] published a version of the algorithm that only uses real coefficients and is aimed at 4×4 real-valued matrix multiplication.

This paper presents, to the best of the author's knowledge, the first hardware implementation of the algorithm proposed in [15], and makes the circuit available as an open-source implementation at [16] for comparison and further development.

The proposed circuit is optimized for hardware implementation with no loss of precision in computation. It is synthesized in a 14 nm technology and its electrical performances are compared with the implementation of the Naive algorithm varying the bit width of the input elements from 4 bit to 64 bit. The proposed circuit, due to the reduced number of binary multiplications, achieves greater area efficiency for input widths above 40 bits but the increased number of add/subtractions results in higher power and delay compared to the Naive implementation. However the power and delay gap decrease as the operand size grows suggesting a potential for higher precision applications.

The paper is organized as follows. Section II presents the Rank-48 algorithm, section III and IV the hardware implementations, Section V the performance comparison, and Section VI the conclusions.

II. Rank-48 matrix multiplication algorithm

Assume that A and B are 4×4 real-valued matrices and we want to compute $C = A \cdot B$, where C is also a 4×4 real-valued matrix. The Naive algorithm given in (1) has a rank of 64 since it requires 64 multiplications. It also needs 48 additions.

$$C_{i,j} = \sum_{k=1}^{4} A_{i,k}\, B_{k,j}, \quad 1 \le i, j \le 4. \tag{1}$$

Recently an algorithm to multiply two 4×4 complex-valued matrices requiring only 48 scalar multiplications was found

PNRR-POC-2022 12376833. Project CAREMODE. Funded by the EU. Next Generation EU. PNRR M6C2. 2.1 Improvement of the biomedical research of the SSN. CUP: D43C22005080006.

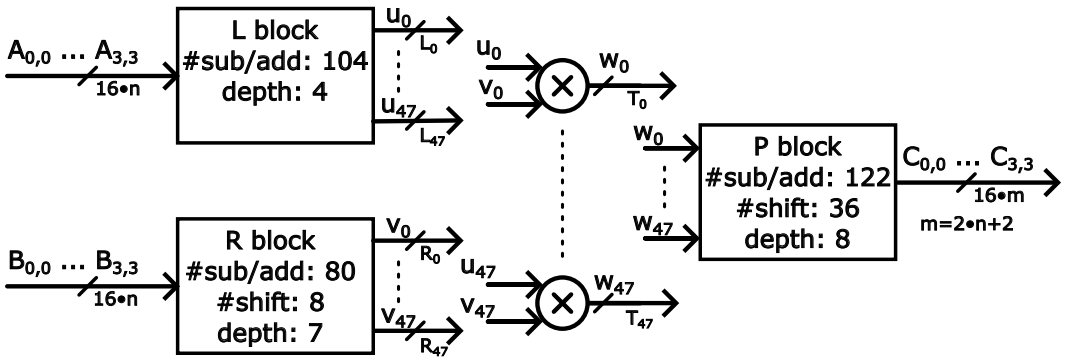

Fig. 1. Schematic of the proposed 4×4 real-valued matrix multiplier based on the Rank-48 modified AlphaEvolve algorithm. L, R, and P blocks use multi-operand addition/subtraction trees; R and P also include shift operations for multiplications and division by power of two.

using a pipeline of large language models orchestrated by an evolutionary coding agent [12]. In [15] a variant of this algorithm that only uses real and rational coefficients aimed at the multiplication of real-valued matrices has been proposed. The mathematical techniques exploited in [15] are outside the scope of this paper but the resulting algorithm can be summarized as follows.

The 4×4 input matrices A, B are vectorized obtaining 16×1 vectors. These are multiplied by real-valued and rational matrices L and R whose dimensions are 48×16. The results are the 48×1 u and w vectors shown in (2) where supescripts indicate the dimensions of the matrices and the vectors.

$$u^{(48\times1)} = L^{(48\times16)} \cdot \mathrm{vec}(A)^{(16\times1)} \tag{2a}$$

$$v^{(48\times1)} = R^{(48\times16)} \cdot \mathrm{vec}(B)^{(16\times1)} \tag{2b}$$

Vectors u and w are multiplied element by element implementing the 48 scalar multiplications that determine the Rank-48 of the algorithm. The resulting vector is multiplied by the 16×48 matrix P. The result is reshaped to get the output matrix C.

$$w^{(48\times1)} = u \odot v \tag{3a}$$

$$c^{(16\times1)} = P^{(16\times48)} \cdot w^{(48\times1)} \tag{3b}$$

$$C^{(4\times4)} = \mathrm{reshape}(c, 4, 4) \tag{3c}$$

The algorithm described above clarifies that the research effort in [15] allowed the determination of the L, R, and P matrices whose complete form is available at [17]. It is worth noting that the entries in the L, R, and P matrices are rational numbers limited to the set $\{0, \pm1, \pm\frac{1}{2}, \pm\frac{1}{4}, \pm\frac{1}{8}\}$ allowing an efficient hardware implementation as will be shown in Section III. In addition, the authors exploited an optimization algorithm to convert the matrix multiplications in (2), (3a) in a straight line program (SLP) consisting in a pseudo code without loops nor conditional branches that is well suited for an hardware implementation (see Section 4 of [15]).

III. Circuit Design and Implementation of the Rank-48 Matrix Multiplication Algorithm

The hardware implementation of the algorithm follows the straight-line program (SLP) reported in [15].

A. Circuit Architecture

The schematic of the proposed circuit is shown in Fig. 1. It is a fully parallel, combinational circuit that relies on a wide datapath to make all input data available simultaneously. This assumption is feasible on modern CPU architectures that support dedicated instruction sets for matrix operations [7]. Moreover, the fully combinational design allows an assessment of the intrinsic performance of the algorithm without the limitations introduced by pipelining or memory bottlenecks.

Fig. 1 shows the three multi-operand add/subtraction trees that realize the transformations defined by the L, R, P matrices. The trees differ both in the number of add/subtraction operations (104, 80, and 122 for the L, R, P matrices, respectively) and in their logic depths (4, 7, and 8 levels, respectively). Shift operations, 8 in the R block and 36 in the P block, implement multiplications and divisions by powers of two as required by the SLP formulation of the algorithm. Finally, Fig. 1 also shows the 48 multipliers of the Rank-48 algorithm and highlights the signal bit-widths, which, as detailed in Sect. III-B, vary across internal signals.

B. Bit Width of the Signals

The elements of the input matrices A and B in Fig. 1 are integers represented in two's complement on n bits taking values in $\left[-2^{n-1}, 2^{n-1}-1\right]$. Each entry of the output matrix C is obtained as the sum of four products; to prevent overflow, the output elements must be represented on $m = 2n + 2$ bits.

In this paper the target is a circuit that provides exact results with no loss of precision. To achieve this, also the bit widths of internal signals must be carefully decided: too few bits can cause overflow, while too many increase area and complexity. Since the ranges of internal signals in the multi-operand add/subtraction trees and the multipliers are difficult to be derived in closed form, a simulation-based approach has been adopted.

A software model of the algorithm has been run 1.2×10^6 times with different bit-widths for the input matrices storing the observed ranges of all the internal signals. This allowed to determine the required number of bits for the signals. For example, signal L_0 in Fig. 1 requires $n + 4$ bits, R_0 requires

$n+1$ bits, while the largest signals, two internal signals in the P block, require $2n+7$ bits. The final bit-width allocation was validated by simulating the synthesized circuit and verifying that outputs matched the software model.

C. Hardware description

The circuit implementing the SLP algorithm described in [15] has been described in SystemVerilog using the bit widths obtained in Section III-B and the schematic in Fig. 1.

The HDL description is parameterized by n, the number of bits of the elements in the input matrices. The arithmetic operations of addition, subtraction, and multiplication are described using the Verilog operators $+$, $-$, and $\star$, allowing the synthesizer to optimize the arithmetic based on the available technology. The HDL source code, the testbench, and the files with the test vectors are available at [16].

IV. Comparison Circuit

The proposed circuit implementing the real-valued AlphaEvolve algorithm for 4×4 matrix multiplication has been compared with a Naive implementation that performs row-by-column multiplication as in (1).

The choice of the Naive algorithm as a comparison is justified since previous works [5] showed that other algorithms, although theoretically more efficient, often struggle to deliver the expected speed-up and heavily rely on the target architecture and the design technique.

Also the Naive algorithm has been implemented with full precision (i.e. using bit-width of the signals that avoid overflow) and in a fully combinational version for a fair comparison with the proposed circuit. The Naive implementation has been described in SystemVerilog, and the source code, which can be used as a reference, is available at [16].

V. Electrical Performances and Benchmarking

The proposed and the circuit for the Naive algorithm have been implemented in a VLSI standard cell flow to evaluate electrical performance. The circuits have been designed varying the number of bits of the input elements of the matrices from 4 to 64 bits.

The circuits are synthesized in a 14 nm FinFET standard cell technology (0.8 V supply, regular V_T) using Cadence Genus with timing constraints of $5\,\mathrm{ns}$ for the Naive implementation, that is faster, and $10\,\mathrm{ns}$ for the proposed circuit. Post synthesis verification has been performed by simulating the netlists with approximately 3300 input vectors and comparing outputs with the expected results. The test vectors, available at [16], include both hand crafted corner cases and randomly generated inputs.

Power dissipation is estimated by simulating the synthesized netlists and collecting the switching activity over 10^4 random input vectors.

The electrical performances show that in most cases the reduced number of binary multiplications required by real valued AplhaEvolve algorithm is counterbalanced by the increased number of additions and subtractions that increase the power dissipation and the delay of the circuit.

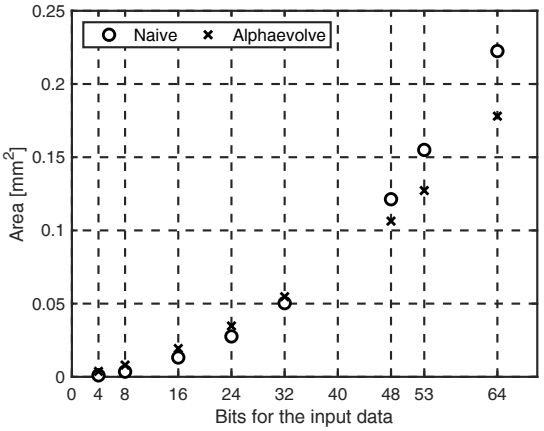

Fig. 2. Naive vs. real-valued AlphaEvolve. Silicon area occupation at varying bit-widths of the input elements

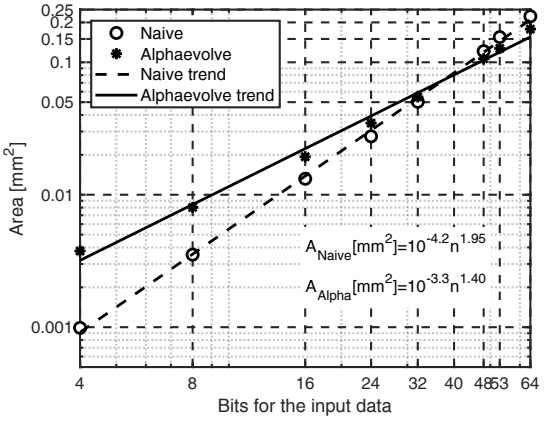

Fig. 3. Silicon area versus input bit-width, comparing Naïve and AlphaEvolve implementations with trend lines.

In terms of silicon area occupation the results favor the Naive implementation for lower bit widths. When the number of input bits increases, due to the square dependence of the area occupation of the multipliers with the number of bits, the proposed circuit overcomes the Naive implementation as shown in Fig. 2. Fig. 3 reports the area occupation in a log-scale with the trend lines that are asymptotically better for the proposed circuit and result in a crossing point around 40 bits. Fig. 2 shows that at 64 bits the Naive implementation requires 25% more hardware resources ($0.222\,\mathrm{mm}^2$ vs. $0.178\,\mathrm{mm}^2$) while at 8 bits the situation is reversed with the Naive implementation requiring 45% less hardware.

In terms of speed, the Naive implementation achieves a shorter delay. It has been synthesized with a timing constraint of $5\,\mathrm{ns}$, and the timing analyzer reported zero slack at 48, 53, and 64 bits. By contrast, the proposed real-valued AlphaEvolve implementation was synthesized with a constraint of $10\,\mathrm{ns}$, resulting in positive slack values of 1541, 832, and $21\,\mathrm{ps}$ at 48, 53, and 64 bits, respectively. These results are detailed in Table I and indicate that the Naive circuit can reliably operate at $200\,\mathrm{MHz}$, whereas the proposed design is limited to approximately $100\,\mathrm{MHz}$.

TABLE I
TIMING SLACK (IN PS) FOR NAIVE AND ALPHAEVOLVE IMPLEMENTATIONS AT DIFFERENT BIT-WIDTHS.

# Bits	Naive ($T = 5$ns)	AlphaEvolve ($T = 10$ns)
4	4345 ps	7127 ps
8	3556 ps	6456 ps
16	2614 ps	5242 ps
24	1830 ps	3708 ps
32	796 ps	3070 ps
48	0 ps	1541 ps
53	0 ps	832 ps
64	0 ps	21 ps

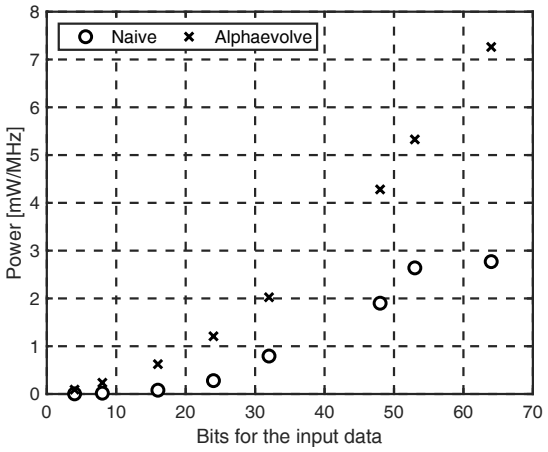

Fig. 4. Naive vs. real-valued AlphaEvolve. Power dissipation at varying bit-widths of the input elements.

Power dissipation results are in Fig. 4 and show that the Naive implementation outperforms the proposed circuit across all bit-widths. This is likely due to the shorter critical path of the Naive implementation, which reduces the propagation of switching activity. However, the percentage difference decreases as the bit-width increases. For instance, at 4 bits, the AlphaEvolve implementation dissipates approximately 30 times more power than the Naive, while at 24, 32, 48, and 53 bits, the gap reduces to 4.3, 2.5, 2.2, and 2.0, respectively (at 53 bits we have $5.3\,\mathrm{mW/MHz}$ and $2.6\,\mathrm{mW/MHz}$ for proposed and Naive circuit, respectively), consistent with the observed hardware requirements. At 64 bits, this trend slightly reverses, as the Naive implementation dissipates only marginally more power than at 53 bits, likely due to the synthesizer being better optimized for standard designs and for bit-widths that are powers of two. The results could change for implementations that are not fully combinational.

VI. CONCLUSION

The paper has presented a circuit implementation of the 4×4 matrix multiplication circuit that implements a recently proposed algorithm derived from the AlphaEvolve finding and adapted for real valued matrix multiplication. The algorithm is innovative being the first one, after 56 years of research in the field, that allows the computation to be carried out use 48 binary multiplications. To the best of author's knowledge, this is the first proposed circuital implementation of the algorithm.

The presented circuit is fully combinational and follows the straight line program implementation proposed by the authors of the algorithm and has been compared against an implementation of the Naive algorithm for 4×4 matrix multiplication.

The description of the circuit is open sourced allowing further developments and comparison with the proposed implementation.

REFERENCES

[1] A. Petropoulos and T. Antonakopoulos, "A scalable FPGA architecture with adaptive memory utilization for GEMM-based operations," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 8, pp. 2334–2338, 2025.

[2] C. Wang, P. Song, H. Zhao, F. Zhang, J. Wang, and L. Zhang, "High-utilization GPGPU design for accelerating GEMM workloads: An incremental approach," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2024, pp. 1–5.

[3] J. Dongarra and P. Luszczek, "Matrix multiplication on digital signal processors and hierarchical memory systems," in *High Performance Computing*. Springer, 1998, pp. 333–343.

[4] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, pp. 161–170, 2015.

[5] T. E. Pogue and N. Nicolici, "Strassen multisystolic array hardware architectures," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 5, pp. 1323–1333, May 2025.

[6] A. Jain, S. Dhar, D. Raghunathan, V. Kathail, and S. Sarkar, "Mxcore: Unified programmable edge matrix processor," *ACM Trans. Archit. Code Optim.*, vol. 19, no. 3, pp. 1–25, 2022.

[7] Intel Corporation, "Intel Advanced Matrix Extensions (Intel AMX)," https:/en.wikipedia.org/wiki/Advanced_Matrix_Extensions, accessed: 2025-08-10.

[8] V. Strassen, "Gaussian elimination is not optimal," *Numerische Mathematik*, vol. 13, pp. 354–356, 1969. [Online]. Available: https://api.semanticscholar.org/CorpusID:121656251

[9] S. Winograd, "On multiplication of 2 × 2 matrices," *Linear Algebra and Its Applications*, vol. 4, no. 4, pp. 381–388, Oct. 1971.

[10] E. Karstadt and O. Schwartz, "Matrix multiplication, a little faster," in *Proc. ACM Symp. Parallelism Algorithms Architectures*, Jul. 2017, pp. 101–110.

[11] O. Schwartz and N. Vaknin, "Pebbling game and alternative basis for high performance matrix multiplication," *SIAM J. Sci. Comput.*, vol. 45, no. 6, pp. C277–C303, 2023.

[12] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, and P.-S. H. et al., "AlphaEvolve: A coding agent for scientific and algorithmic discovery," 2025. [Online]. Available: https://arxiv.org/abs/2506.13131

[13] G. DeepMind, "AlphaEvolve: Mathematical results," 2025. [Online]. Available: https://colab.research.google.com/github/google-deepmind/alphaevolve_results/blob/master/mathematical_results.ipynb

[14] F. Bakreski, "AlphaEvolve-matrixmul-verification," 2025, accessed: 2025-08-10. [Online]. Available: https://github.com/PhialsBasement/AlphaEvolve-MatrixMul-Verification

[15] J.-G. Dumas, C. Pernet, and A. Sedoglavic, "A non-commutative algorithm for multiplying 4×4 matrices using 48 non-complex multiplications," *arXiv preprint arXiv:2506.13242*, June 16 2025.

[16] E. Napoli, "HW implementation of a real-valued alphaevolve 4×4 matrix multiplication algorithm," https://github.com/etnapoli/HW-implementation-real-valued-AlphaEvolve-matrix-mul, 2025, accessed: Sep. 15, 2025.

[17] J.-G. Dumas, B. Grenet, C. Pernet, and A. Sedoglavic, "Plinopt: A collection of C++ routines handling linear & bilinear programs," https://github.com/jgdumas/plinopt, Jan. 2024.